

Self-Attention is a fundamental building block of modern deep learning models, especially within the Transformer architecture. It works by dynamically computing attention weights that reflect how relevant each element in a sequence is to every other element, thereby capturing long-range dependencies across the entire input. Unlike conventional recurrent neural networks, self-attention supports fully parallel computation, which greatly accelerates training and has led to its broad adoption across fields like natural language processing and computer vision.

In this framework, the query matrix is denoted as $(Q \in \mathbb{R}^{n \times d_k})$, and the key matrix as $(K \in \mathbb{R}^{n \times d_k})$ (note: typically both (Q) and (K) have the same shape; the transpose is applied during computation).

The self-attention mechanism proceeds through several key operations:

1. **Compute attention scores**: The similarity between all pairs of elements is measured by taking the dot product of the query and key matrices:

$$S = QK^T \quad \text{(tag1)}$$

Here, $(S \in \mathbb{R}^{n \times n})$ is the raw attention score matrix, where each entry (S_{ij}) quantifies how much the (i) -th position should attend to the (j) -th position.

2. **Scale and normalize**: To mitigate the issue of large dot products causing gradients to vanish or explode during training, the scores are scaled by $(\sqrt{d_k})$ and then passed through a Softmax function to yield a probability distribution over the sequence:

$$A = \text{Softmax}\left(\frac{S}{\sqrt{d_k}}\right) \quad \text{(tag2)}$$

The resulting matrix $(A \in \mathbb{R}^{n \times n})$ contains normalized attention weights that sum to 1 across each row.

3. **Compute weighted output**: Finally, these attention weights are used to form a context-aware representation by taking a weighted sum of the value vectors, where the value matrix is $(V \in \mathbb{R}^{n \times d_v})$:

$$O = AV \quad \text{(tag3)}$$

The output $(O \in \mathbb{R}^{n \times d_v})$ thus incorporates global contextual information from the full input sequence, with each position updated based on its relevance to all others.

> **Note**: The original statement defined $(K \in \mathbb{R}^{d_k \times n})$, which is unconventional. In standard formulations, both (Q) and (K) are $(n \times d_k)$, and

the product uses $\backslash (K^{\text{top}} \backslash)$. The corrected version above reflects this widely accepted convention.